- Frontend Architecture Explanation
  - 1. Frontend Overview
  - 2. App Entry and Global Shell
  - 3. Routing Architecture
  - 4. Page Renderer Architecture
    - 4.1 Landing Page
    - 4.2 Login Page
    - 4.3 Dashboard Page
    - 4.4 Students Page
    - 4.5 Student Detail Page
    - 4.6 Analytics Page
    - 4.7 Interventions Page
    - 4.8 Counselors Page
  - 5. Service Layer
    - 5.1 Student Service
    - 5.2 Intervention Service
    - 5.3 Realtime Service
    - 5.4 Session/Auth Service
    - 5.5 API Configuration
  - 6. Utility and Business Logic Layer
    - 6.1 Risk Calculator
    - 6.2 Analytics Engine
  - 7. Dashboard Architecture Using Code
  - 8. Students Page Architecture Using Code
  - 9. Student Detail Architecture Using Code
  - 10. Analytics Architecture Using Code
  - 11. Interventions Page Architecture Using Code
  - 12. Counselors Page Architecture Using Code
  - 13. Realtime Frontend Architecture
  - 14. State Management Architecture
    - 14.1 localStorage
    - 14.2 sessionStorage
    - 14.3 In-memory page state
  - 15. Styling Architecture
  - 16. End-to-End Frontend Flow
  - 17. Best Viva Summary
  - 18. Best Teacher-Style Answers

- Where is routing written?
- Where is app startup written?
- Where is login logic written?
- Where is student filtering written?
- Where is risk calculation written?
- Where are dashboard analytics written?
- Where are API calls written?
- Where are charts written?
- Where is page-state persistence written?
- Where are realtime notifications written?

Frontend Architecture Explanation

This document explains the frontend architecture of the project using the actual code files used in implementation, so it can be used for presentation, viva, and documentation.

1. Frontend Overview

The frontend is built as a **Single Page Application (SPA)** using:

- **Vite**
- **Vanilla JavaScript**
- **HTML**
- **CSS**
- **Chart.js**
- **Socket.IO Client**

The application loads one main HTML page and then updates the visible page dynamically using JavaScript and hash-based routing.

Main frontend folders:

- `frontend/src/router`
- `frontend/src/renderers`
- `frontend/src/services`
- `frontend/src/utils`
- `frontend/styles`

2. App Entry and Global Shell

The common frontend shell is defined in:

- `frontend/index.html`

This file contains:

- global header
- navigation bar
- main app container with id `app`
- global footer
- CSS imports
- the JavaScript entry point

The application startup logic is written in:

- `frontend/src/main.js`

Important code responsibilities in `main.js`:

- route change listener:
  - `window.addEventListener('hashchange', renderPage);`
- initial page render:
  - `renderPage();`
- theme initialization:
  - `initTheme()`
- auth-based header/nav update:
  - `checkAuthState()`
- realtime authentication sync:
  - `syncRealtimeAuth({ token, user })`

What this means architecturally:

- `index.html` gives the permanent structure of the frontend
- `main.js` acts as the startup controller for the application

If asked "where is app startup written?", answer:

- It is written in `frontend/src/main.js`, where `renderPage()` is called and global event listeners are attached.

---

3. Routing Architecture

Routing is implemented in:

- `frontend/src/router/router.js`

The routing function is:

- `renderPage()`

This function:

- reads `window.location.hash`
- decides which page renderer to call
- applies access control before rendering

Implemented routes:

- `#/home`
- `#/login`
- `#/dashboard`
- `#/students`
- `#/student/:id`
- `#/analytics`
- `#/interventions`
- `#/counselors`

Renderer mapping is written directly inside `renderPage()`:

- `renderLanding()`
- `renderLogin()`
- `renderDashboard()`
- `renderStudents()`
- `renderStudentDetail(id)`
- `renderAnalytics()`
- `renderInterventions()`

- `renderCounselors()`

Security inside routing:

- authentication guard for private routes
- admin guard for counselors page

Navigation UI logic:

- `setActiveNav(hash)` updates the active navigation link

What this means architecturally:

- `router.js` is the navigation layer
- it controls which page appears in the single app container
- it also controls route-level access

If asked "where is routing written?", answer:

- Routing is written in `frontend/src/router/router.js` inside `renderPage()`.

If asked "where is route protection written?", answer:

- Route protection is also inside `renderPage()` in `frontend/src/router/router.js`.

---

4. Page Renderer Architecture

Each page is implemented in a separate renderer file inside:

- `frontend/src/renderers`

This is the main page/component design layer of the frontend.

4.1 Landing Page

File:

- `frontend/src/renderers/landing.js`

Main function:

- `renderLanding()`

Responsibilities:

- render public home page
- show project overview
- show CTA buttons depending on login state

4.2 Login Page

File:

- `frontend/src/renderers/login.js`

Main function:

- `renderLogin()`

Responsibilities:

- render login form
- submit credentials to backend
- store `token` and `user` in `localStorage`
- redirect to dashboard after successful login

4.3 Dashboard Page

File:

- `frontend/src/renderers/dashboard.js`

Main function:

- `renderDashboard()`

Responsibilities:

- fetch students and interventions
- compute KPIs and risk summaries
- render dashboard cards, watchlist, and charts

4.4 Students Page

File:

- `frontend/src/renderers/students.js`

Main function:

- `renderStudents()`

Responsibilities:

- show full student roster
- apply filters and search
- render priority list
- open student detail page

4.5 Student Detail Page

File:

- `frontend/src/renderers/studentDetail.js`

Main function:

- `renderStudentDetail(id)`

Responsibilities:

- fetch one student
- show case review
- show risk chart and intervention timeline
- save intervention
- assign counselor for admin users

4.6 Analytics Page

File:

- `frontend/src/renderers/analytics.js`

Main function:

- `renderAnalytics()`

Responsibilities:

- render full analytics center
- apply filters
- manage drill-down state
- manage simulator
- render multiple charts

4.7 Interventions Page

File:

- `frontend/src/renderers/interventions.js`

Main function:

- `renderInterventions()`

Responsibilities:

- show intervention queue
- select student for action
- log intervention
- show intervention history

4.8 Counselors Page

File:

- `frontend/src/renderers/counselors.js`

Main function:

- `renderCounselors()`

Responsibilities:

- show counselor cards

- show current assignments
- assign unassigned students to counselors

What this means architecturally:

- every page has its own module
- responsibilities are separated clearly
- routing and page rendering are not mixed together

If asked "where is the Students page code written?", answer:

- The Students page structure and logic are written in `frontend/src/renderers/students.js` inside `renderStudents()`.

---

5. Service Layer

The service layer is inside:

- `frontend/src/services`

It handles all communication with the backend and shared session logic.

5.1 Student Service

File:

- `frontend/src/services/studentService.js`

Important functions:

- `enrichStudent(student)`
- `getAllStudents()`
- `getStudentById(id)`

Responsibilities:

- fetch student data from backend
- calculate enriched fields before UI rendering
- return frontend-ready student objects

5.2 Intervention Service

File:

- `frontend/src/services/interventionService.js`

Important functions:

- `saveIntervention(...)`
- `getInterventionsForStudent(studentId)`
- `getAllInterventions()`

Responsibilities:

- save intervention records
- fetch intervention history
- handle backend validation errors

5.3 Realtime Service

File:

- `frontend/src/services/realtimeService.js`

Important functions:

- `syncRealtimeAuth({ token, user })`
- `disconnectRealtime()`
- `clearUnreadAlerts()`

Responsibilities:

- connect to Socket.IO server
- listen for live events
- update alert badge
- show toast notifications

5.4 Session/Auth Service

File:

- `frontend/src/services/authSession.js`

Important function:

- `handleUnauthorized(response)`

Responsibilities:

- detect expired sessions
- clear auth state
- redirect to login page

5.5 API Configuration

File:

- `frontend/src/config/api.js`

Important exports:

- `API_BASE_URL`
- `SOCKET_URL`

Responsibilities:

- centralize API base URL
- centralize socket server URL

What this means architecturally:

- all backend communication is separated from the page renderer files
- renderers call service functions instead of writing all fetch logic inline

If asked "where are API calls written?", answer:

- API calls are written in the service layer, such as `getAllStudents()` in `frontend/src/services/studentService.js` and `saveIntervention()` in `frontend/src/services/interventionService.js`.

---

6. Utility and Business Logic Layer

The business logic of the application is separated into:

- `frontend/src/utils/riskCalculator.js`
- `frontend/src/utils/analyticsEngine.js`

6.1 Risk Calculator

File:

- `frontend/src/utils/riskCalculator.js`

Important functions:

- `calculateRiskScore(student)`
- `getRiskStatus(score)`
- `getRiskFactors(student)`
- `calculateDropoutProbability(riskScore)`
- `generateRiskTrend(baseRisk)`

Responsibilities:

- calculate numeric risk score
- convert score to `high`, `medium`, `low`
- determine the visible risk factors
- estimate dropout probability
- generate trend data for chart display

6.2 Analytics Engine

File:

- `frontend/src/utils/analyticsEngine.js`

Important functions:

- `getDriverMatches(student)`
- `filterData(students, filters)`
- `computeKPIs(students)`
- `riskDrivers(students)`
- `areaPressureIndex(students)`

- `interventionPriorities(students)`
- `buildCohortNarrative(students)`
- `simulateAttendance(students, improvementPct)`
- `generateInsights(students)`

Responsibilities:

- calculate dashboard metrics
- generate analytical summaries
- determine intervention priority areas
- support filters and drill-down analysis
- power the analytics page charts and insights

What this means architecturally:

- business logic is reusable and independent
- page files stay cleaner
- logic can be reused by dashboard, analytics, students, and student detail pages

If asked "where is risk calculated?", answer:

- It is calculated in `frontend/src/utils/riskCalculator.js` inside `calculateRiskScore(student)`.

If asked "where are KPIs and analytics generated?", answer:

- They are generated in `frontend/src/utils/analyticsEngine.js`.

---

7. Dashboard Architecture Using Code

File:

- `frontend/src/renderers/dashboard.js`

The dashboard is built mainly inside:

- `renderDashboard()`

How the code works:

1. It first shows a loading state

2. It fetches students and interventions together using `Promise.all`
3. It calculates:
    - KPIs
    - cohort narrative
    - intervention priorities
    - area pressure
    - risk drivers
4. It renders the full dashboard UI with `app.innerHTML`
5. It creates charts using Chart.js

Important chart functions:

- `renderPressureChart(pressure)`
- `renderDriverChart(drivers)`

If asked "how is the dashboard implemented?", answer:

- The dashboard page is implemented in `frontend/src/renderers/dashboard.js`. Data is fetched using `Promise.all`, analytics are computed using utility functions, and charts are rendered with Chart.js.

---

8. Students Page Architecture Using Code

File:

- `frontend/src/renderers/students.js`

Main helper functions:

- `buildRows(students, currentUserId)`
- `buildMyRoster(students, currentUserId)`
- `buildPriorityList(students)`
- `buildInsight(filtered)`
- `loadSavedFilters()`
- `saveFilters(filters)`

Main page function:

- `renderStudents()`

How the code works:

1. Fetches all students using `getAllStudents()`
2. Renders toolbar, filters, table, priority list, and side summary
3. Attaches event listeners for:
    - student row click
    - action button click
    - filter change
    - search input
    - clear button
4. Applies filters using the internal `applyFilters()` function
5. Saves and restores filter state using `sessionStorage`

If asked "where is the filter logic written?", answer:

- It is written in `applyFilters()` inside `frontend/src/renderers/students.js`.

If asked "where is page-state persistence written?", answer:

- It is written in `loadSavedFilters()` and `saveFilters()` inside `frontend/src/renderers/students.js`.

---

9. Student Detail Architecture Using Code

File:

- `frontend/src/renderers/studentDetail.js`

Main functions:

- `renderStudentDetail(id)`
- `renderDetail(app, student)`
- `buildActionPlan(student, interventions)`
- `buildInterventionTimeline(interventions)`

- `buildDriverCards(drivers)`
- `buildRecommendedOptions(student, actionPlan)`
- `renderTrendChart(student)`

How the code works:

1. `renderStudentDetail(id)` fetches the required student
2. `renderDetail(app, student)` builds the full case review UI
3. It fetches interventions for that student
4. It creates a retention strategy using `buildActionPlan`
5. It renders the risk trend chart
6. It loads staff assignment data for admin users
7. It saves intervention entries through the intervention service
8. It stores page draft state in `sessionStorage`

Student detail state persistence:

- `loadStudentDetailState(studentId)`
- `saveStudentDetailState(studentId, state)`

If asked "where is counselor assignment implemented?", answer:

- It is implemented inside `frontend/src/renderers/studentDetail.js`, where staff are fetched and the assignment update action is attached for admin users.

If asked "where is the intervention save button logic written?", answer:

- It is written in the click handler attached to `#saveIntervention` inside `frontend/src/renderers/studentDetail.js`.

---

10. Analytics Architecture Using Code

File:

- `frontend/src/renderers/analytics.js`

This is the most advanced renderer in the frontend.

State object at top of file:

- `state.all`
- `state.filters`
- `state.charts`
- `state.drillDown`
- `state.simulatorValue`

Important state functions:

- `loadSavedAnalyticsState()`
- `persistAnalyticsState()`
- `getFiltered()`
- `setDrillDown(type, value)`
- `syncFilterUI()`
- `onFilterChange()`
- `updateAllCharts()`

Important chart functions:

- `updateRiskChart(data)`
- `updateAreaChart(data)`
- `updateDriverChart(data)`
- `updateGradeChart(data)`
- `updatePressureChart(data)`
- `updateRadarChart(data)`
- `updateEconomicChart(data)`
- `updateAttendanceChart(data)`

Other important logic:

- `updateKPIs(data)`
- `updateNarrative(data)`
- `updatePriorityList(data)`
- `updateDriverTable(data)`
- `updateFactorBars(data)`
- `updateInsights(data)`
- `updateSimulator()`
- `updateDrillBanner()`

How the code works:

1. Loads all students

2. Loads saved analytics page state
3. Renders the analytics page shell
4. Attaches filter, reset, and slider listeners
5. Applies filters and drill-downs
6. Builds charts dynamically with Chart.js
7. Updates chart theme when light/dark mode changes

If asked "where are chart codes written?", answer:

- They are written in `frontend/src/renderers/analytics.js` in separate chart update functions.

If asked "where is analytics page state written?", answer:

- It is written in `loadSavedAnalyticsState()` and `persistAnalyticsState()` in `frontend/src/renderers/analytics.js`.

---

11. Interventions Page Architecture Using Code

File:

- `frontend/src/renderers/interventions.js`

Important functions:

- `getStudentRecommendations(student)`
- `buildPriorityQueue(students)`
- `buildQueueCards(students)`
- `buildPrioritySummary(priorities)`
- `buildHistoryHTML(interventions)`
- `buildTypeOptions(recommendedTypes)`
- `buildSelectionSummary(student)`
- `loadSavedInterventionState()`
- `saveInterventionPageState(...)`

Main page function:

- `renderInterventions()`

How the code works:

1. Loads students and interventions using `Promise.all`
2. Creates a priority queue of students needing support
3. Renders intervention form and recent history
4. Lets user choose student from queue or dropdown
5. Suggests intervention type based on student condition
6. Saves intervention through `saveIntervention(...)`
7. Stores selected student, intervention type, and draft note in `sessionStorage`

If asked "where is intervention queue logic written?", answer:

- It is written in `buildPriorityQueue(students)` inside `frontend/src/renderers/interventions.js`.

If asked "where is draft intervention form persistence written?", answer:

- It is written in `loadSavedInterventionState()` and `saveInterventionPageState(...)` inside `frontend/src/renderers/interventions.js`.

---

12. Counselors Page Architecture Using Code

File:

- `frontend/src/renderers/counselors.js`

Main function:

- `renderCounselors()`

How the code works:

1. Fetches counselor list from backend
2. Fetches students
3. Calculates student assignment per counselor
4. Renders staff cards dynamically
5. Adds click actions for:

    - opening student detail
    - assigning unassigned students

If asked "where is the admin staff management code written?", answer:

- It is written in `frontend/src/renderers/counselors.js`.

---

13. Realtime Frontend Architecture

File:

- `frontend/src/services/realtimeService.js`

Main logic:

- establish socket connection
- authenticate socket with token
- listen for backend events
- show alerts and toasts

Important event handlers:

- `student_assigned`
- `intervention_logged`
- `student_unassigned`

How it is connected to global app logic:

- `main.js` calls `syncRealtimeAuth({ token, user })` when user is logged in
- `disconnectRealtime()` is called on logout or no-auth state

If asked "where are live notifications written?", answer:

- They are written in `frontend/src/services/realtimeService.js`.

---

14. State Management Architecture

The frontend does not use Redux or another framework store.
Instead, it uses browser storage and local page state.

14.1 `localStorage`

Used for:

- auth token
- logged-in user
- theme preference

Written mainly in:

- `frontend/src/main.js`
- `frontend/src/renderers/login.js`

14.2 `sessionStorage`

Used for:

- Students page filters
- Analytics page filters, drill-down, simulator
- Interventions page selected student and draft note
- Student detail page draft form and assignment selection

Written in:

- `frontend/src/renderers/students.js`
- `frontend/src/renderers/analytics.js`
- `frontend/src/renderers/interventions.js`
- `frontend/src/renderers/studentDetail.js`

14.3 In-memory page state

Used especially in:

- `frontend/src/renderers/analytics.js`

through the `state` object.

If asked "how is frontend state managed?", answer:

- Session and auth data are managed using `localStorage`, page interaction state is preserved using `sessionStorage`, and temporary page state is managed inside renderer-level variables and objects.

---

15. Styling Architecture

Styling files:

- `frontend/styles/global.css`
- `frontend/styles/dashboard.css`
- `frontend/styles/forms.css`
- `frontend/styles/landing.css`

Responsibilities:

- global design system
- dashboard layout and widgets
- forms and controls
- landing page visuals
- light/dark mode support

Login page also uses Tailwind via CDN from:

- `frontend/index.html`

What this means architecturally:

- shared visual system is separated from page logic
- theme changes are handled through CSS variables and body classes

---

16. End-to-End Frontend Flow

The frontend flow can be explained like this:

1. Browser loads `frontend/index.html`
2. `frontend/src/main.js` starts the app

3. `frontend/src/router/router.js` reads the route
4. Router calls the correct renderer
5. Renderer requests data from service layer if needed
6. Service layer fetches backend data
7. Utility layer computes risk and analytics
8. Renderer injects HTML into the `#app` container
9. Event listeners are attached
10. State is stored using `localStorage` or `sessionStorage`
11. Realtime updates are handled through Socket.IO

---

17. Best Viva Summary

Use this short answer:

> My frontend follows a layered SPA architecture. `index.html` provides the common shell, `main.js` handles startup and global state, and `router.js` handles hash-based navigation and route protection. Each page is implemented in a separate renderer file. API communication is separated into service files, and business logic such as risk calculation and analytics is implemented in utility files. Session state is managed using `localStorage` and `sessionStorage`, and live updates are handled through Socket.IO.

---

18. Best Teacher-Style Answers

Where is routing written?

- `frontend/src/router/router.js`

Where is app startup written?

- `frontend/src/main.js`

Where is login logic written?

- `frontend/src/renderers/login.js`

Where is student filtering written?

- `frontend/src/renderers/students.js`

Where is risk calculation written?

- `frontend/src/utils/riskCalculator.js`

Where are dashboard analytics written?

- `frontend/src/utils/analyticsEngine.js`

Where are API calls written?

- `frontend/src/services/studentService.js`
- `frontend/src/services/interventionService.js`

Where are charts written?

- `frontend/src/renderers/dashboard.js`
- `frontend/src/renderers/analytics.js`
- `frontend/src/renderers/studentDetail.js`

Where is page-state persistence written?

- `frontend/src/renderers/students.js`
- `frontend/src/renderers/analytics.js`
- `frontend/src/renderers/interventions.js`
- `frontend/src/renderers/studentDetail.js`

Where are realtime notifications written?

- `frontend/src/services/realtimeService.js`